Factors, dates, and strings

EVR 628- Intro to Environmental Data Science

Juan Carlos Villaseñor-Derbez (JC)

Table of contents

Exercise 0 - Set up **1**

Exercise 1 - Dates **2**

Exercise 2 - Strings **3**
Part 1 - Data contained in a column with strings of text 3
Part 2 - Oh no! Data are STILL contained in a column with strings of text! 4

Exercise 3 - Visualization (a bit of everything) **5**

Prompt:

> You are given a logbook of fauna sightings during whale watching tours from two vessels operating in the Revillagigedo Archipelago (A.K.A. the Socorro Islands), Mexico. You are about to embark on a tagging expedition, so you want to make sure you get on the right boat. Your goal is therefore to: a) Identify which vessel has the most sightings, and b) Identify the most frequent species sighted.

Exercise 0 - Set up

Post-it up

0. Add the outline snippet to your snippets.[1]
1. Download this csv file to your `EVR 628` project folder. Save it into `data/raw/`.
2. Start a new script, call it `factors_dates_and_strings.R` and save it into your `scripts/01_processing/` folder

[1] Hint: Tools > Edit Code Snippets... Remember to use tabs, not spaces

3. Add a code outline and load the `tidyverse` and `janitor` packages at the top of your script

Post-it down

```
## SET UP ###############################################################################
# Load packages ----------------------------------------------------------------
library(janitor)
library(tidyverse)
```

Exercise 1 - Dates

Post-it up

1. Read your data into an object called `tour_data <-` and clean the column names
2. Inspect your data. How many columns / rows? What are these columns?
3. Create a new object called `tour_data_clean <-` and add a new column called `date` that contains the date. We will build a large pipeline starting here.
4. Remove the `year`, `month`, and `day` columns that we no longer need
5. Place the `date` column all the way to the left

```
# Load data  -------------------------------------------------------------------
tour_data <- read_csv("data/raw/tour_data.csv") |>
  clean_names()

dim(tour_data)
colnames(tour_data)

## PROCESSING ###########################################################################
# Add a date -------------------------------------------------------------------
tour_data_clean <- tour_data |>                    # 3) New object
  mutate(date = make_date(year, month, day)) |>    # 4) Build my date
  select(date, everything(), -c(year, month, day)) # 5) Remove and organize columns
```

```
[1] 60  6
```

```
[1] "year"       "month"      "day"        "vessel"     "passengers"
[6] "notes"
```

```
The data look like this now:
```

```
# A tibble: 6 x 4
  date       vessel passengers notes
  <date>     <chr>       <dbl> <chr>
1 2025-01-01 Condor         21 8 Sea lions; 5 Whales; 4 Whale shark
2 2025-01-07 Falcon          6 5 whale; 7 Sea lions; 5 Sea turtle; 4 sea turtle
3 2025-01-13 Falcon          8 5 Dolphins
4 2025-01-19 Condor          6 4 Sea turtle; 4 sea turtle
5 2025-01-25 Condor         29 2 sea turtle; 8 whale
6 2025-02-01 Falcon         19 3 Whale shark; 7 whale
```

Post-it down

Exercise 2 - Strings

Part 1 - Data contained in a column with strings of text

Post-it up

1. As demonstrated in class, use `separate_longer_delim()` to create one row per species mentioned in the data
2. Standardize the data so all strings appear in lowercase
3. Make sure they are all singular

```
## PROCESSING #########################################################################
# Add a date ---------------------------------------------------------------------
tour_data_clean <- tour_data |>
  mutate(date = make_date(year, month, day)) |>
  select(date, everything(), -c(year, month, day)) |>
  separate_longer_delim(cols = notes, delim = ";") |>  # 1) Separate column into new rows
  mutate(notes = str_to_lower(notes),                  # 2) All strings as lower case
         notes = str_remove(notes, "s$"))              # 3) All are singular now

cat("The data look like this now:")
```

```
The data look like this now:
```

```
head(tour_data_clean)
```

```
# A tibble: 6 x 4
  date       vessel passengers notes
  <date>     <chr>       <dbl> <chr>
1 2025-01-01 Condor         21 "8 sea lion"
2 2025-01-01 Condor         21 " 5 whale"
3 2025-01-01 Condor         21 " 4 whale shark"
4 2025-01-07 Falcon          6 "5 whale"
5 2025-01-07 Falcon          6 " 7 sea lion"
6 2025-01-07 Falcon          6 " 5 sea turtle"
```

Post-it down

Part 2 - Oh no! Data are STILL contained in a column with strings of text!

Post-it up

1. Look at the documentation for `str_extract()`
2. Run the first two examples at the bottom directly in your console. What seems to be going on?

pause to discuss

3. Using the `str_extract()` function, build a new column called `fauna` that contains the species observed[2]
4. Print your data to the console. Does `fauna` look right?
5. Using the `str_extract()` function, build a new column called `n` that contains the number of organisms observed (make sure it's numeric)
6. Remove the `notes` column

```r
## PROCESSING #################################################################
# Add a date ------------------------------------------------------------------
tour_data_clean <- tour_data |>
  mutate(date = make_date(year, month, day)) |>
  select(date, everything(), -c(year, month, day)) |>
  separate_longer_delim(cols = notes, delim = ";") |>
  mutate(notes = str_to_lower(notes),
         notes = str_remove(notes, "s$"),
         fauna = str_extract(string = notes, pattern = "[a-z ]+$"), # 3) Extract fauna
         fauna = str_squish(fauna),                                 # 4) Remove white space
         n = str_extract(string = notes, pattern = "[:digit:]+"),   # 5) Extract n
         n = as.numeric(n)) |>                                            # and make sure it
```

[2]Hint: Look at the documentation for `?regex`

```
  select(-notes)                                                    # 6) Remove notes

cat("The data look like this now:")
```

```
The data look like this now:
```

```
head(tour_data_clean)
```

```
# A tibble: 6 x 5
  date       vessel passengers fauna           n
  <date>     <chr>       <dbl> <chr>       <dbl>
1 2025-01-01 Condor         21 sea lion        8
2 2025-01-01 Condor         21 whale           5
3 2025-01-01 Condor         21 whale shark     4
4 2025-01-07 Falcon          6 whale           5
5 2025-01-07 Falcon          6 sea lion        7
6 2025-01-07 Falcon          6 sea turtle      5
```

Post-it down

Exercise 3 - Visualization (a bit of everything)

Post-it up

1. Build a new data frame called `tour_data_viz` that contains the following columns[3]:

- `vessel`
- `fauna`
- `n_per_trip`

2. Recreate the plot below

Post-it down

[3]Hint, you will need two `group_bys`, one `mutate`, and one `summarize`

```r
## Build tour_data_viz ---------------------------------------------------------
tour_data_viz <- tour_data_clean |>
  group_by(vessel) |>
  mutate(effort = n_distinct(date)) |>
  group_by(vessel, fauna) |>
  summarize(n_per_trip = sum(n / effort),
            .groups = "drop")

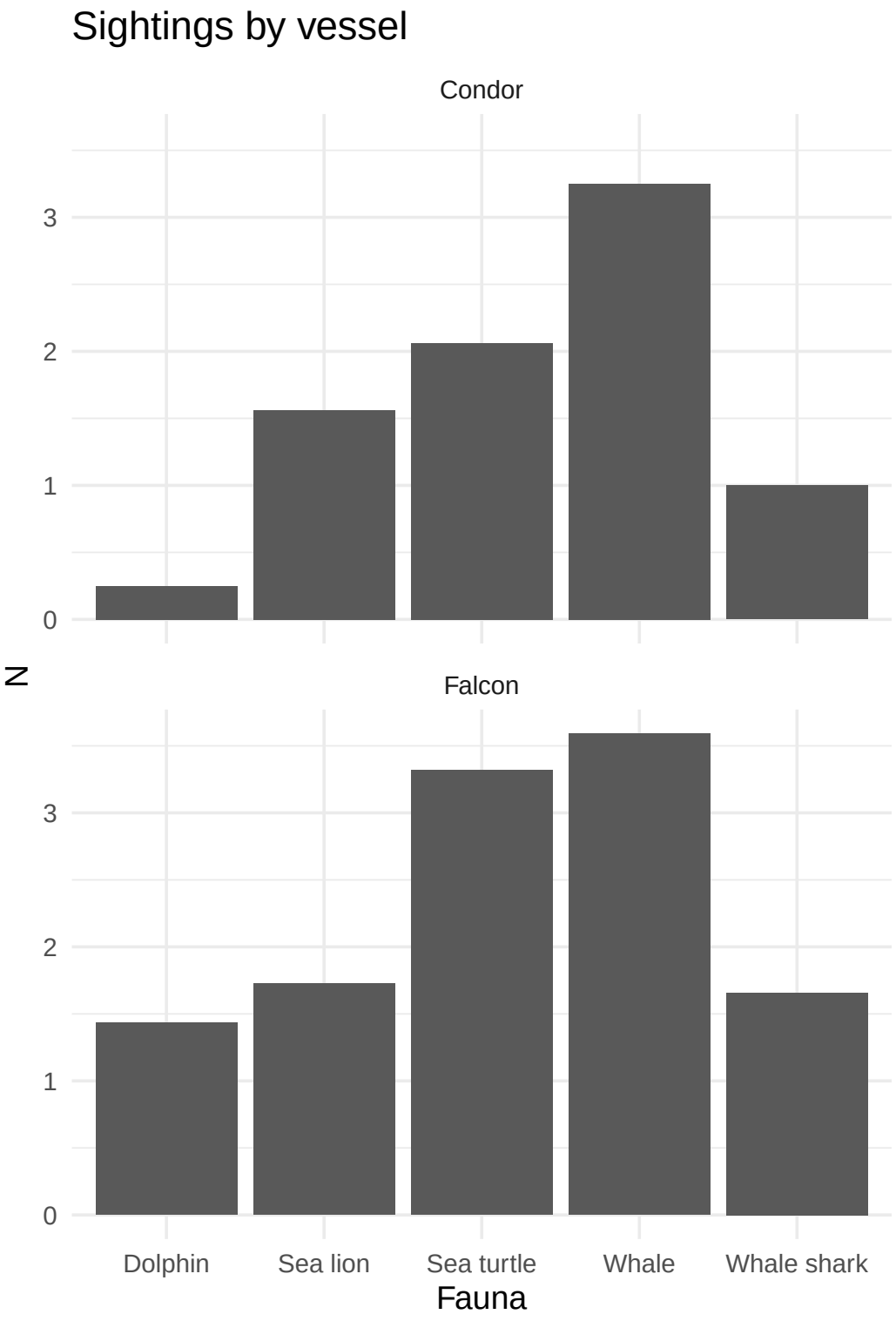
cat("The data look like this now:")
```

```
The data look like this now:
```

```r
head(tour_data_viz)
```

```
# A tibble: 6 x 3
  vessel fauna       n_per_trip
  <chr>  <chr>            <dbl>
1 Condor dolphin           0.25
2 Condor sea lion          1.56
3 Condor sea turtle        2.06
4 Condor whale             3.25
5 Condor whale shark       1
6 Falcon dolphin           1.43
```

```r
## VISUALIZE ######################################################################
ggplot(data = tour_data_viz,
       mapping = aes(x = str_to_sentence(fauna), # This is the only str-relevant change here
                     y = n_per_trip)) +
  geom_col() +
  facet_wrap(~vessel, ncol = 1) +
  labs(title = "Sightings by vessel",
       x = "Fauna",
       y = "N") +
  theme_minimal(base_size = 14)
```


Sightings by vessel
Condor
Falcon
N
0
1
2
3
Dolphin
Sea lion
Sea turtle
Whale
Whale shark
Fauna